

12 Greedy Algorithms

In this chapter we'll look at an approach for solving problems with a recursive structure which is faster than dynamic programming when it works, but works on a more limited set of problems. In Section 12.1 we'll consider the interval partitioning problem, where we assign classes to classrooms, and in Section 12.2 we'll look at scheduling jobs on a computer in a way that minimizes the maximum lateness. Finally, in Section 12.3 we'll discuss the general pattern for greedy algorithms, and contrast it with the dynamic programming pattern that we learned about last chapter.

12.1 Interval Partitioning

Let's start by examining a problem faced by universities like Texas A&M. We are given a list of classes, each of which has a start time and an end time. We want to assign the classes to the classrooms. Classes cannot be assigned to the same classroom if their meeting times overlap. Since space at the university is limited, we would like to minimize the total number of classrooms that we use.

In the real world there are a variety of additional constraints. For example, classes have different numbers of students and classrooms have different capacities. Some classes meet only one day a week, while others meet on two or three. Some classes' instructors may have preferences for particular rooms. For the sake of keeping the problem simple to analyze, we will ignore all of these issues in this section and focus only on the constraint that we cannot have two classes meeting at the same time in the same room.

More formally, in the interval partitioning problem our input is an array $A[1, \dots, n]$ of pairs of positive integers, representing start times and finish times for classes. We say that two classes $c_i = (s_i, f_i)$ and $c_j = (s_j, f_j)$ *conflict* if $s_i \leq s_j < f_i$ or $s_j \leq s_i < f_j$ (i.e. if one of them starts while the other is ongoing). Our goal is to find the minimum number of classrooms that we need to be able to assign these classes to rooms such that no classes which conflict are assigned to the same classroom.

This is a problem that we could definitely solve using the recursive backtracking technique from Chapter 10. We simply iterate through the list of classes, and for each one we try assigning it to each possible classroom (including creating a new classroom), recursively assigning the rest of the classes in a way that doesn't conflict.

But while it's easy to design a correct recursive backtracking algorithm, it's not clear how to design one that will convert into an efficient dynamic programming algorithm. In particular, it's not clear what short summary of our choices so far we could use to keep the number of subproblems small.

But what if we could speed up the algorithm in a different way? What if instead of trying out each class in every available classroom, we just arbitrarily choose one, without recursively checking how many rooms we'll need for the rest of the classes. Maybe things will work out anyway?

We call this approach "greedy", because we are making a sequence of small choices (like in recursive backtracking or DP algorithms), but instead of looking at the long-term consequences of every possible choice, we just make a choice that seems to make sense based on what we've seen so far, without even looking at the rest of the array.

The greedy algorithm we just discussed is given below. We iterate through the classes and assign each one to the first room where it has no conflicts. If it has a conflict with every classroom, we create a new room for the class.

```

GREEDYALG( $A[1, \dots, n]$ )
  rooms  $\leftarrow$  an empty 2-D array of classes
  For  $i$  from 1 to  $n$ :
    For  $j$  from 1 to size(rooms)+1:
      If  $j > \text{size}(\text{rooms})$ :
        Append an array containing only  $A[i]$  to rooms
      Else if  $A[i]$  does not conflict with any class in  $\text{rooms}[j]$ :
        Add  $A[i]$  to  $\text{rooms}[j]$ 
      Break
  Return size(rooms)

```

Theorem 58. GREEDYALG does not correctly solve the interval partitioning problem.

Proof. Suppose that $A = \{(1, 4), (8, 10), (3, 7), (6, 9)\}$. Since GREEDYALG assigns each class to the first room that it fits in, it will start by assigning $(1, 4)$ to room 1. $(8, 10)$ does not conflict with $(1, 4)$, so it will also be assigned to room 1. Then, since $(3, 7)$ conflicts with $(1, 4)$, the algorithm will create a second room to assign it to. Finally since $(6, 9)$ conflicts with both $(8, 10)$ and $(3, 7)$, the algorithm will create a third room to assign it to and will return 3.

In fact, however, it is possible to fit these four classes into two rooms. If we assign $(1, 4)$ and $(6, 9)$ to room 1 and $(3, 7)$ and $(8, 10)$ to room 2, then neither room has a conflict. $\square$

As the previous example illustrates, intuitive-sounding greedy algorithms can frequently fail, returning incorrect answers for certain inputs. Even if we cannot come up with a specific example where a greedy algorithm fails, we should not trust it unless we can give a proof that it will always return the correct answer. This is true for all algorithms, but it's particularly important for greedy algorithms, because they are especially prone to sounding “obviously” correct even when they don't work.

In this case, our failed greedy algorithm is actually salvageable. As the bad example in our incorrectness proof shows, a failure mode of GREEDYALG is scheduling two classes in the same room with an awkward gap in-between them, too small to fit another class. The way to avoid this is to sort the classes and consider them in order of increasing start time.

Sorting the classes by start time also allows us to make the algorithm more efficient. If we know that class i starts later than any class we have scheduled so far, then to determine if class i can be scheduled in room j , it's sufficient to check if class i starts after the end of the last class we scheduled in room j . Since all the other classes in room j must start earlier than the last class we scheduled there, if class i does not conflict with the last class scheduled in the room, it cannot conflict with any of the others either.

Based on these observations, we give the following improved algorithm:

```

GREEDYINTERVALPARTITION( $A[1, \dots, n]$ )
  Sort  $A$  by start time
  lastClass  $\leftarrow$  an empty array of integers
  For  $i$  from 1 to  $n$ :
    For  $j$  from 1 to size(lastClass)+1:
      If  $j > \text{size}(\text{lastClass})$ :
        Append  $A[i].\text{finish}$  to lastClass
        Break
      Else if  $A[i].\text{start} \geq \text{lastClass}[j]$ :
        lastClass[j]  $\leftarrow A[i].\text{finish}$ 
        Break
  Return size(lastClass)

```

Theorem 59. For every array of classes A , GREEDYINTERVALPARTITION returns the minimum number of classrooms needed to schedule these classes without conflicts. Furthermore, GREEDYINTERVALPARTITION runs in $O(n^2)$ time.

Proof. We note that $\text{lastClass}[j]$ stores the finish time of the last class that we scheduled for room j . Thus, our inner loop finds the first room where we can schedule class $A[i]$ without a conflict

and assigns $A[i]$ there. Therefore, when the outer loop finishes, we have found a sequence of room assignments using $\text{size}(\text{lastClass})$ rooms with no conflicts.

It remains to prove that it is not possible to assign the classrooms of A using fewer rooms than our algorithm does. Let k be the number of rooms that GREEDYINTERVALPARTITION uses. Let c_k be the first class assigned to room k . In order for this to happen, it must have been the case that when we were searching for a room for c_k , every room $1 \leq i < k$ had a class c_i with $c_i.\text{finish} > c_k.\text{start}$. Furthermore, since we sorted the classes by start time, every class considered before c_k must start at the same time as c_k or earlier, so we have $c_i.\text{start} \leq c_k.\text{start}$ for every $i \in \{1, \dots, k-1\}$.

Therefore, at the time $c_k.\text{start}$, all k of the classes $c_1, \dots, c_k$ are taking place. This means that no two of these classes can be scheduled in the same room, so every valid assignment of these classes to rooms must use at least k rooms.

Finally, we analyze the runtime of our algorithm. Sorting A takes $\Theta(n \log n)$ time (e.g. using mergesort). Our outer for loop runs $O(n)$ times and our inner for loop runs $O(k)$ times. The work inside of the inner for loop is $O(1)$, so the total time for the outer for loop is $O(nk)$. Thus, the overall runtime of our algorithm is $O(\max\{n \log n, nk\})$. Since $\log n \in O(n)$ and $k \in O(n)$ (we cannot do worse than scheduling every class in its own room), this runtime is $O(n^2)$. $\square$

12.2 Job Scheduling

Let's see another variety of scheduling problem which can be solved using a greedy algorithm. In the single machine scheduling problem we are given a set of jobs J which our machine needs to run. Each job j has two attributes:

- $j.\text{time}$ is the time that j takes our machine to complete (measured e.g. in minutes)
- $j.\text{deadline}$ is the time by which we are supposed to complete j (measured in minutes since the time when we start running jobs).

There are a variety of questions we could ask in this setting. e.g. we can ask whether it is possible to sequence the jobs in a way that satisfies all of the deadlines. We could seek to finish as many jobs on time as possible. We could seek to minimize the average lateness. The objective we will focus on in this section, however, is minimizing the *maximum* lateness.

We will assume that we have our machine running jobs non-stop until all of them are finished. So the choice that we need to make is what order to process the jobs in. i.e. we need to choose an ordering $j_1, j_2, \dots, j_n$ for the jobs. Given such a sequence, the time at which we finish job j_i is

$$t_i := \sum_{k=1}^i j_k.\text{time},$$

the total runtime of j_i plus all of the jobs that we run before it. The lateness of job j_i is

$$\ell_i := \max(0, t_i - j_i.\text{deadline})$$

i.e. it is the difference between when we complete the job and the deadline, assuming that we complete the job late. Our goal is to find a sequence for the jobs which minimizes $\max_{i=1}^n \ell_i$.

For example, if our jobs (represented by (time, deadline) pairs) are $A = (1, 7)$, $B = (5, 2)$, $C = (3, 3)$, $D = (20, 5)$ and we choose to process the jobs in the order A, B, C, D (from shortest time to longest), then they finish at times $t_1 = 1, t_2 = 6, t_3 = 9, t_4 = 29$, and their lateness is $\ell_1 = 0, \ell_2 = 4, \ell_3 = 6, \ell_4 = 24$, for a maximum lateness of 24. A better ordering would have been B, C, D, A , which gives us finishing times of $t_1 = 5, t_2 = 8, t_3 = 28, t_4 = 29$ with lateness $\ell_1 = 3, \ell_2 = 5, \ell_3 = 23, \ell_4 = 22$, for a maximum lateness of 23.

This time, the appropriate algorithm is one familiar to procrastinating students everywhere: “focus on whichever job has the earliest deadline”. That is, our algorithm is just to sort the jobs from earliest deadline to latest, so simple that even giving pseudocode would be superfluous. The interesting step is proving that this algorithm actually does keep the maximum lateness as small as possible.

Theorem 60. *For any set of jobs J , the maximum lateness is minimized by processing the jobs in order from earliest deadline to latest.*

Proof. We prove this by induction on n .

- Base case: If $n = 1$, then there is only one possible ordering, so ordering by deadline performs the same as any other ordering and is therefore optimal.
- Induction Hypothesis: Suppose that for every set of fewer than n jobs, ordering them from smallest to largest deadline minimizes the maximum lateness.
- Induction Step: Let J be a set of $n > 1$ jobs, let $j_1, \dots, j_n$ be these jobs ordered from earliest to latest deadline. Let $L := \max_{i=1}^n \ell_i$ be the maximum lateness of these jobs. We consider two cases:
 - First suppose that $L = \ell_n$, i.e. that j_n is a job with maximum lateness in our schedule. In this case, $L = t_n - j_n.\text{deadline} = (\sum_{i=1}^n j_i.\text{time}) - j_n.\text{deadline}$. We note that no matter how we order the jobs, the last job will always finish at the same time ($\sum_{i=1}^n j_i.\text{time}$). Thus, the lateness of the last job will always be the sum of the runtimes of all the jobs minus the deadline of the last job. Since every other job has a deadline $\leq j_n.\text{deadline}$, any other job will have a lateness of at least L when processed last. Since some job must be processed last, this means that every ordering has maximum lateness at least L , and thus our ordering minimizes the maximum lateness.
 - Now suppose that $L \neq \ell_n$, i.e. that the last job is not the latest one. Then $L = \max_{i=1}^{n-1} \ell_i$. By our induction hypothesis, any ordering of $j_1, \dots, j_{n-1}$ has maximum lateness at least L . Thus, for any ordering of $j_1, \dots, j_n$, if we remove j_n , the maximum lateness is at least L . Adding j_n back cannot decrease the maximum lateness (it can only delay jobs not cause them to finish earlier), so therefore the original schedule of n jobs also had maximum lateness at least L . Thus, every ordering of the n jobs has maximum lateness at least L , and thus our ordering minimizes the maximum lateness.

This proof relied on an *exchange argument* to show that our choice to make the job with the latest deadline last was optimal. We noted that exchanging the job that we put last with any other job could not improve the maximum lateness. Thus, our choice of the last job was optimal, and by induction, our choice of the remaining jobs was also optimal. This kind of argument occurs commonly in analyzing greedy algorithms. □

12.3 Greedy Algorithms vs Dynamic Programming

The exact definition of a greedy algorithm is a bit fuzzy, and it's not important to be able to classify whether or not a particular algorithm counts as "greedy". As a general rule of thumb, however, greedy algorithms are ones that:

- Make small incremental decisions without backtracking
- Make locally-optimal (greedy) decisions, without considering the global picture
- Often follow simple fixed priority rules

For example, our algorithm for interval partitioning assigned the classes to rooms one at a time (making small incremental decisions), without looking ahead at what classes were coming next (making locally-optimal decisions) following the simple rule to evaluate classes in order from earliest to latest start time.

To recognize when a greedy algorithm applies, you should look for two key characteristics: optimal substructure and a greedy choice property.

Optimal substructure is actually a requirement not only for greedy algorithms, but also for dynamic programming and recursive backtracking ones. This is the property that an optimal solution can be built by making a single choice and then optimally solving a smaller version of the same problem (or a generalized version of the problem, since for both DP algorithms and greedy algorithms it's often necessary to keep some summary of our choice so far). For example, an optimal room assignment for n classes can be built by choosing a room for the first class, and then choosing an optimal assignment for the remaining $n - 1$ classes (given an array of times when each room becomes available). A maximum-lateness-minimizing ordering for n jobs can be found by assigning the job with the last deadline to the final position, and then finding a maximum-lateness-minimizing ordering for the remaining $n - 1$ jobs.

The advantage of greedy algorithms over dynamic programming is that while dynamic programming relies on overlapping subproblems, so that the total number of possible subproblems is small (polynomial), greedy algorithms can work even when the number of possible subproblems is large, because we do not have to explore all of them.

The greedy choice property says that a globally optimal solution can be reached by making locally optimal (greedy) choices. This is the part that can require a clever proof, often built on induction and some kind of exchange argument. The problems that we gave dynamic programming algorithms for in the previous chapter have optimal substructure (e.g. a longest increasing subsequence for A can be built by adding an element onto a longest increasing subsequence for some subarray of A), but do not have this greedy choice property.

Note that to be able to prove the greedy choice property, we need to be using the right greedy choice rule. As we saw in Section 12.1, it's easy to give a natural looking greedy choice rule that does not correctly solve the problem.

12.4 Comprehension Questions

1. Consider the sequence of classes $(10, 15), (1, 7), (2, 9), (14, 18), (8, 13), (3, 11)$. How many classrooms do we need to schedule these classes?
2. Suppose that we have jobs with the following (time, deadline) pairs. How should we sequence them to minimize the maximum lateness? $A = (12, 9), B = (8, 10), C = (14, 7), D = (5, 11)$
3. True or false? If `GREEDYINTERVALPARTITION( $A$ )` returns k , then there is some point in time at which k classes from A are all meeting simultaneously.
4. Consider the following algorithm for job scheduling: "Order the jobs from shortest time to longest time". Give a counterexample to prove that this greedy algorithm does not necessarily minimize the maximum lateness.
5. True or false? Every problem that can be solved in polynomial time with dynamic programming can also be solved in polynomial time with a greedy algorithm?

12.5 Exercises

1. For the third chapter in a row, you are planning a road trip between cities A and B . They are still connected by a long straight highway. This time, however, you are rich, and don't care at all about how much you spend on gas. Instead, your most valuable resource is time, and so your goal is to minimize the number of times that you need to stop for fuel.

Suppose that you are given d , the distance in miles that your car can travel on a full tank of gas, as well as a *sorted* array $G[1, \dots, n]$, where $G[i]$ is the location of the i th gas station, given as a distance in miles from your starting point. Assume that your trip begins at the first gas station (so $G[1] = 0$) with an empty tank of gas, and ends at the n th gas station (where you do not need to refuel). Assume that it is possible to complete the trip (i.e. there is no stretch of $> d$ miles with no gas stations). Give an $O(n)$ time algorithm to calculate the minimum number of times that you will need to refuel in order to complete this trip.

Make sure to prove that your algorithm is correct, i.e. that the number of stations you return really is the minimum possible.

2. You are the sole employee at a call center. n different customers have just called in at the same time. Fortunately, you have supernatural powers to help you do your job. Before picking up the phone, you can immediately tell how long it will take you to handle each of these calls. You have created an array $A[1, \dots, n]$, where $A[i]$ is the time that it will take to handle the i 'th call. You would like to choose an ordering $i_1, \dots, i_n$ in which to take the calls, which will minimize the total waiting time of your customers. Note that the total wait time is

$$\sum_{j=1}^n \sum_{k=1}^{j-1} A[i_k].$$

That is, the total wait time is the sum of the wait times of all the customers, and the wait time of a customer is the total time you spend helping everyone whose call you take before theirs.

Give an $O(n \log n)$ time algorithm to find an ordering for the calls which minimizes the total wait time.

3. We define a balanced string of parentheses recursively as follows:
 - ε (the empty string) is a balanced string of parentheses
 - If s is a balanced string of parentheses then so is (s)
 - If s and t are balanced strings of parentheses, then so is st (the concatenation of s and t).

For example, the string $((()))$ is a balanced string of parentheses because it is created by applying the third rule to $((()))$ and $()$. $()$ is balanced because it is created by applying the second rule to ε (which is balanced by the first rule), and $((()))$ is balanced because it is created by applying the second rule to $()$. On the other hand $)()$ is not a balanced string of parentheses since it cannot be constructed according to these rules.

Suppose that you are given an array $A[1, \dots, n]$ where each element of A is either (or).

- (a) Give an $O(n)$ -time algorithm to determine whether or not A represents a balanced string of parentheses.
 - (b) Give an $O(n)$ -time algorithm to find the length of the longest subsequence of A which is a balanced string of parentheses.
4. Suppose that you start with the number $x = 1$, and you may repeatedly choose between incrementing x (increasing its value by 1) and tripling x (multiplying its value by 3). Your goal is to reach a target number y using as few of these steps as possible. For example, if y is 411, then the optimal number of steps is 9, and the nine step sequence to get x from 1 to 411 is:

$$1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 15 \rightarrow 45 \rightarrow 135 \rightarrow 136 \rightarrow 137 \rightarrow 411.$$

Give an algorithm which takes as input a positive integer y and outputs the minimum number of steps to obtain y (starting from $x = 1$) using this process. The running time of your algorithm should be polynomial in the length of the input (i.e. your running time should be $O((\log y)^c)$ for some constant c). Make sure to prove the correctness of your algorithm.

5. In the class selection problem, we are given as input an array A of n classes, each with a start time and an end time, and the goal is to choose a maximum size subset of the classes such that no two are running at the same time (i.e. for any distinct classes x and y which are chosen, either $x.end \leq y.start$, or $y.end \leq x.start$).

In this problem, we consider 5 different greedy algorithms for this problem. For each of the following algorithms, either *prove* that it always returns an optimal schedule, or give a concrete

example (i.e. a list of start and end times) on which the algorithm does not generate an optimal schedule. Assume that each algorithm has the base cases of “If there are no classes, return the empty schedule”, and “If there is only one class, choose that class.” Assume that ties are broken arbitrarily (i.e. in a manner outside of your control). You do not need to analyze the runtime of these algorithms.

- (a) Choose the class x that starts last, discard the classes that conflict with x , and recurse.
 - (b) Choose the class x that ends last, discard the classes that conflict with x , and recurse.
 - (c) If no classes conflict, choose them all. Otherwise, discard the class x with the longest duration and recurse.
 - (d) If no classes conflict, choose them all. Otherwise, discard the class x which conflicts with the most other classes and recurse.
 - (e) If any class x is completely contained in another class y (i.e. y starts before x and ends after x) then discard y and recurse. Otherwise, choose the class z that ends last, discard the classes that conflict with z and recurse.
6. Suppose that you are given a base-10 positive integer x with n -digits as well as a number $k < n$, and you want to return the smallest number that can be obtained by removing exactly k digits from x . e.g. if $x = 1432219$ and $k = 3$, you should return 1219. Give an $O(n)$ time algorithm for this problem.
7. An *Egyptian fraction* is a sum of a finite sequence of fractions which each have numerator 1, and which have *distinct* positive integer denominators e.g. $\frac{1}{3} + \frac{1}{10} + \frac{1}{17}$. Give an algorithm which takes as input a rational number $r = p/q$, where $p, q \in \mathbb{Z}^+$ with $p < q$, encoded as a numerator-denominator pair. Your algorithm should return an Egyptian fraction which is equal to r . Your Egyptian fraction should use at most p total unit fractions. For example, if $r = 5/6$, you could return $1/2 + 1/3$. You could also return $1/2 + 1/4 + 1/12$. You could not return $1/6 + 1/6 + 1/6 + 1/6 + 1/6$, because the denominators of these fractions are not distinct. Make sure to prove the correctness of your algorithm.

Fun fact 1: The earliest known algorithm for representing an arbitrary rational number as an Egyptian fraction was developed in 1202 by Fibonacci (the same mathematician after whom the Fibonacci numbers are named).

Fun fact 2: While this problem only asks you to represent p/q with an Egyptian fraction using at most p unit fractions, it is possible to give an algorithm that uses $O(\log \log q)$ unit fractions. This was proven in a September 2026 OpenAI preprint.